APLICAÇÃO DE JSON-WEB-TOKENS - MEU CONFEITO

AMBIENTE BACK-END

O Back-end do projeto será uma API RESTful, a qual responderá às requisições da aplicação Front-end, em React Native. O framework escolhido para a API foi o Express.js, de versão superior à 5.2, baseado em Node.js e adaptado para trabalhar com o superset de JavaScript, TypeScript, a fim de modernizar a estrutura do código, bem como auxiliar em seu desenvolvimento.

ABORDAGEM DE TOKENS

No Node.js, o módulo responsável por prover as ferramentas necessárias para manipular os JSON-WEB-TOKENS (JWTs) é fornecido pela biblioteca 'jsonwebtoken'. Os tokens, tanto de acesso quanto de atualização, serão gerados por meio da biblioteca padrão do node, 'node:crypto', e a chave de assinatura destes será gerada aleatoriamente de forma externa.

A abordagem escolhida, em função da segurança e aplicabilidade, para a gestão de tokens foi a geração de dois destes distintos: o **token de acesso** e o **token de atualização**.

FUNCIONAMENTO DOS TOKENS

O token de acesso será primeiramente gerado a partir da ação de login do usuário, sendo enviado no corpo de resposta e, pelo Front, guardado e enviado no Header das próximas requisições. Este terá **tempo de vida** curto, **de 30 minutos**, e mediará todas todas as requisições que necessitam de autenticação, através de um middleware que verifica a sua assinatura na API. No payload deste token, serão guardadas informações importantes sobre quem o gerou (e, consequentemente, quem fez a requisição), como o ID de usuário e o seu nível de acesso (cargos), que serão utilizados para outras importantes verificações de segurança na API. Após a expiração, que retornará um erro de autenticação a ser manipulado pelo Front, um novo token de acesso poderá ser gerado a partir do token de atualização, caso este esteja válido.

O token de atualização será, também, gerado a partir do login, mas será enviado no corpo da resposta apenas caso o cliente da requisição não seja uma aplicação Web, devido ao cuidado que deve ser tomado em função da importância deste token. No caso do cliente de requisição ser do tipo **Web**, o token de atualização será enviado como uma variável (`refreshToken`) nos **cookies** do navegador, que contará com opções como `httpOnly`, que bloqueia a manipulação com JavaScript, `sameSite`, que obriga o cookie a ter vindo especificamente do site da aplicação, e `secure`, que, quando a API estiver em modo de produção (efetivamente trabalhando), assegurará que a requisição foi feita utilizando HTTPS. Caso a requisição tenha vindo de uma aplicação local, como **mobile**, então o **token de atualização será enviado no corpo da resposta** e seguramente guardado no armazenamento local (no caso do React Native, o Secure Storage). Este token terá um **tempo de vida** maior, **de 3 meses**, sendo armazenado no banco de dados e guardando os campos `“user_id”` e `“session_id”`**.** Isto significa que haverá **um token de atualização por usuário por sessão**. Quando o token de acesso expirar, uma requisição, disparada pelo próprio Front, enviará o token de atualização armazenado e, caso as informações estejam de acordo com o banco, novos tokens de atualização e de acesso serão gerados, permitindo o uso contínuo da aplicação. Quando o token de atualização expirar, ou quando houver muitas sessões diferentes de um mesmo usuário, na próxima vez que este realizar uma ação que exige autenticação, será necessário efetuar o login novamente (no caso de múltiplos clientes, o mais antigo terá o token apagado), gerando novos tokens de acesso e de atualização.

APLICAÇÃO ATUAL

```
async logIn(data: AuthDTO.LogInRequest) {
    // Lógica do login...

    const payload = {
      id: foundUser.id,
      role: foundUser.role,
    };

    const token = jwt.sign(payload, env.JWT_SECRET, { expiresIn: '30m' });
    const refreshToken = randomBytes(40).toString('hex');

    // Criação do Token de Atualização no Banco...

    const userData: AuthDTO.LogInResponse = {
      user,
      auth: {
        accessToken: token,
        tokenType: 'Bearer',
        expiresIn: 1800,
      },
    };

    if (data.clientType === 'web') {
      const cookie = {
        name: 'refreshToken',
        val: refreshToken,
        options: {
          httpOnly: true,
          secure: env.NODE_ENV === 'production',
          sameSite: 'strict' as 'strict',
          maxAge: env.RT_TTL_DAYS * 24 * 60 * 60 * 1000,
        },
      };

      return { userData, cookie };
    } else {
      // mobile
      userData.auth.refreshToken = refreshToken;

      return { userData };
    }
  }
```

FLUXO DE AUTENTICAÇÃO

